# Proyecto de Lenguajes Formales

FIRST/FOLLOW, analizadores LL(1) y SLR(1), y evaluación de una SDD

Juan José Duque Marín  Juan Lucas Ramírez Romero
Samuel Pulgarin

SI2002 Lenguajes Formales — Universidad EAFIT

Noviembre de 2026

## Contenido

**1 Introducción** **1**

**2 Algoritmos del libro y estructuras de datos** **1**

**3 Ítems 1 y 2 con una gramática propia** **3**
3.1 FIRST y FOLLOW . . . 3
3.2 Tabla LL(1) . . . 4
3.3 Autómata LR(0) y tabla SLR(1) . . . 4
3.4 Una cadena analizada paso a paso . . . 5

**4 Ítem 3: definición dirigida por la sintaxis** **5**
4.1 Gramática . . . 5
4.2 Tabla de atributos . . . 6
4.3 Reglas semánticas . . . 6
4.4 Ejemplo resuelto: la cadena `2:101` . . . 7
4.5 Clasificación . . . 8
4.6 Pruebas . . . 8
4.7 Experimento de circularidad . . . 9

**5 Pruebas de los Ítems 1 y 2** **10**

**6 Problemas encontrados y cómo los resolvimos** **11**

## 1 Introducción

Este informe documenta la implementación, en Python 3 y usando solo la biblioteca estándar, de los algoritmos de las secciones 4.4, 4.5, 4.6, 5.1 y 5.2 de Aho, Lam, Sethi y Ullman [1]. El programa calcula los conjuntos FIRST y FOLLOW (Ítem 1), decide si una gramática es LL(1) y/o SLR(1) y analiza cadenas con el analizador correspondiente (Ítem 2), y evalúa una definición dirigida por la sintaxis (SDD) sobre el árbol de análisis de una cadena mediante el grafo de dependencias (Ítem 3). Las instrucciones de ejecución están en el `README.md`.

## 2 Algoritmos del libro y estructuras de datos

| **Parte** | **Libro** | **Implementación y estructuras de datos** |
|---|---|---|
| Gramática | Convenciones del §2.2 del enunciado | `grammar.py`. `Production` (dataclass inmutable: índice, cabeza, cuerpo como `tuple[str]`; la producción-$\varepsilon$ tiene cuerpo vacío). `Grammar`: lista de producciones, `dict` cabeza $\rightarrow$ producciones, listas ordenadas de no terminales y terminales. |
| FIRST | §4.4.2, reglas 1–3 de FIRST; FIRST de cadenas al final de §4.4.2 | `first_follow.py`. `dict[str, set[str]]`. Iteración de punto fijo: se recorren todas las producciones hasta que ningún conjunto cambia. `first_of_string` implementa FIRST$(X_1 \cdots X_n)$; FIRST$(\varepsilon) = \{\varepsilon\}$. |
| FOLLOW | §4.4.2, reglas 1–3 de FOLLOW | `dict[str, set[str]]`, punto fijo; \$ $\in$ FOLLOW$(S)$. |
| Prueba LL(1) | Definición al inicio de §4.4.3 (tres condiciones) | `ll1.py`, `LL1._check_condition`: para cada par de alternativas se verifican las tres condiciones y se guarda una lista de violaciones legibles. |
| Tabla LL(1) | Algoritmo 4.31 | `dict[(A, a), list[Production]]`; una celda con más de una producción es un conflicto. |
| Analizador LL(1) | Algoritmo 4.34, §4.4.4 | Pila como `list` (tope al final) con \$ y $S$; devuelve la derivación por la izquierda (lista de producciones emitidas) y, si se pide, la traza (Fig. 4.21). |
| Ítems LR(0) | §4.6.2 | `slr.py`. `Item`: (producción, posición del punto), inmutable y *hashable*. Gramática aumentada con $S' \rightarrow S$; $S'$ se representa con la cadena `"S'"` (dos caracteres) para que nunca choque con un no terminal de la entrada. |
| CLOSURE y GOTO | Fig. 4.32 y definición de GOTO en §4.6.2 | CLOSURE recorre una lista que crece mientras se recorre (con un conjunto de ítems vistos). |
| Colección canónica | Fig. 4.33 | Lista de estados (tuplas de ítems) + `dict[frozenset[Item], int]` para reconocer estados repetidos + `dict[(i, X), j]` de transiciones. Los estados se numeran en orden BFS, probando los símbolos en el orden en que aparecen tras el punto; así se reproduce la numeración de la Fig. 4.31. |

| **Parte** | **Libro** | **Implementación y estructuras de datos** |
|---|---|---|
| Tabla SLR(1) | Algoritmo 4.46, §4.6.4 | ACTION: `dict[(i, a), list[Action]]` con acciones `("s", j)`, `("r", prod)`, `("acc", None)`; GOTO: `dict[(i, A), j]`. Los conflictos se guardan como (estado, símbolo, acciones) y se clasifican en desplazamiento/reducción o reducción/reducción (§4.5.4). |
| Analizador SLR(1) | Algoritmo 4.44, §4.6.3 | Pila de estados (y una pila paralela de símbolos solo para la traza, Fig. 4.38). Devuelve la secuencia de reducciones: la derivación por la derecha en orden inverso. |
| SDD como datos | §5.1.1, Fig. 5.1 | `sdd/spec.py`. `AttrRef(pos, nombre)` ($0$ = cabeza, $1..k$ = cuerpo), `Rule(target, deps, fn, texto)`, `SDD` (atributos sintetizados y heredados por símbolo, reglas por producción, atributo principal). Las reglas se escriben con la notación del libro ($R_1$, $N_2\ldots$) y se traducen a posiciones. |
| Validación de la SDD | §5.1.1 y Step 2 del enunciado | `validate`: cada regla define un sintetizado de la cabeza o un heredado de un no terminal del cuerpo, usa solo atributos de su producción, cada atributo requerido se define exactamente una vez y $S$ no tiene heredados. |
| Árbol de análisis | §5.1.2 | `sdd/tree.py`. `Node`: id, símbolo, producción, hijos, `dict` de atributos. Un solo `build_tree` sirve para los dos analizadores: con LL(1) consume la derivación por la izquierda en preorden; con SLR(1) invierte las reducciones (derivación por la derecha) y construye los hijos de derecha a izquierda. |
| Grafo de dependencias | §5.2.1, Fig. 5.7 | `sdd/depgraph.py`. Vértices: pares (id de nodo, atributo); aristas en listas de adyacencia. Para cada nodo interior se instancian las reglas de *su* producción, con una arista desde cada atributo usado hacia el definido. |
| Orden de evaluación | §5.2.2 | Algoritmo de Kahn con un *heap* (desempate por (id, atributo), orden determinista). Si quedan vértices sin procesar hay un ciclo; un DFS iterativo sobre esos vértices extrae un ciclo concreto y se reporta `SDD is circular`. |

| Parte | Libro | Implementación y estructuras de datos |
|---|---|---|
| Clasificación | §5.2.3 y §5.2.4 | `classify`: comprobación automática de S-atribuida y L-atribuida, usada para contrastar la justificación hecha a mano (§4.5). |

# 3 Ítems 1 y 2 con una gramática propia

Usamos una gramática de listas anidadas, como `(i,(i,i),())`, que no aparece en el enunciado:

```
4
S -> (A)
A -> BC e
C -> ,BC e
B -> i S
```

Tiene producciones-$\varepsilon$, recursión indirecta ($S \Rightarrow (A) \Rightarrow (BC) \Rightarrow (SC)$) y FOLLOW que exige varias iteraciones. Todo lo siguiente fue calculado por el programa (`python3 main.py -sets -verbose -trace < examples/report_grammar.txt`).

## 3.1 FIRST y FOLLOW

```
First(A) = {(, i, e}
First(B) = {(, i}
First(C) = {,, e}
First(S) = {(}
Follow(A) = {)}
Follow(B) = {), ,}
Follow(C) = {)}
Follow(S) = {), ,, $}
```

## 3.2 Tabla LL(1)

Las tres condiciones de §4.4.3 se cumplen; por ejemplo, para $A \to BC \mid \varepsilon$: FIRST$(BC) = \{(, i\}$ y FOLLOW$(A) = \{)\}$ son disjuntos. La tabla (Algoritmo 4.31) no tiene entradas múltiples:

```
   (         )       ,         i        $
-  --------  ------  --------  -------  -
A  A -> BC   A -> e             A -> BC
B  B -> S                       B -> i
C            C -> e  C -> ,BC
S  S -> (A)
```

## 3.3 Autómata LR(0) y tabla SLR(1)

La Figura 1 muestra la colección canónica (Fig. 4.33) dibujada por nuestro visualizador; la tabla SLR(1) (Algoritmo 4.46) no tiene conflictos, así que la gramática es LL(1) y SLR(1).

```
state  (   )           ,           i   $           A  B   C   S
-----  --  ----------  ----------  --  ----------  -  --  --  -
0      s2                                          1
```

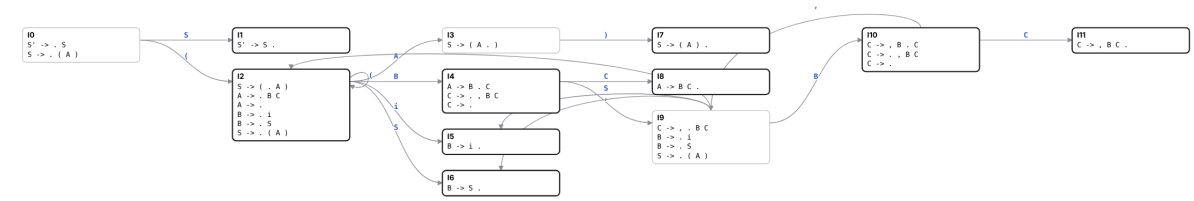



Figura 1: Autómata LR(0) de la gramática de listas anidadas.

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 1 | | | | acc | | | |
| 2 | s2 | r(A -> e) | s5 | | 3 | 4 | 6 |
| 3 | | s7 | | | | | |
| 4 | | r(C -> e) | s9 | | | 8 | |
| 5 | | r(B -> i) | r(B -> i) | | | | |
| 6 | | r(B -> S) | r(B -> S) | | | | |
| 7 | | r(S -> (A)) | r(S -> (A)) | r(S -> (A)) | | | |
| 8 | | r(A -> BC) | | | | | |
| 9 | s2 | | s5 | | 10 | | 6 |
| 10 | | r(C -> e) | s9 | | | 11 | |
| 11 | | r(C -> ,BC) | | | | | |

## 3.4 Una cadena analizada paso a paso

Cadena `(i,())` con el analizador LL(1) (Algoritmo 4.34; formato de la Fig. 4.21):

| MATCHED | STACK | INPUT | ACTION |
|---|---|---|---|
| | S $ | (i,())$ | output S -> (A) |
| | ( A ) $ | (i,())$ | match ( |
| ( | A ) $ | i,())$ | output A -> BC |
| ( | B C ) $ | i,())$ | output B -> i |
| ( | i C ) $ | i,())$ | match i |
| (i | C ) $ | ,())$ | output C -> ,BC |
| (i | , B C ) $ | ,())$ | match , |
| (i, | B C ) $ | ())$ | output B -> S |
| (i, | S C ) $ | ())$ | output S -> (A) |
| (i, | ( A ) C ) $ | ())$ | match ( |
| (i,( | A ) C ) $ | ))$ | output A -> e |
| (i,( | ) C ) $ | ))$ | match ) |
| (i,() | C ) $ | )$ | output C -> e |
| (i,() | ) $ | )$ | match ) |
| (i,()) | $ | $ | accept |

La misma cadena con el analizador SLR(1) (Algoritmo 4.44; formato de la Fig. 4.38):

```
STACK          SYMBOLS  INPUT    ACTION
0                       (i,())$  shift 2
0 2            (        i,())$   shift 5
0 2 5          (i       ,())$    reduce by B -> i
0 2 4          (B       ,())$    shift 9
0 2 4 9        (B,      ())$     shift 2
0 2 4 9 2      (B,(     ))$      reduce by A -> e
0 2 4 9 2 3    (B,(A    ))$      shift 7
0 2 4 9 2 3 7  (B,(A)   )$       reduce by S -> (A)
0 2 4 9 6      (B,S     )$       reduce by B -> S
0 2 4 9 10     (B,B     )$       reduce by C -> e
0 2 4 9 10 11  (B,BC    )$       reduce by C -> ,BC
0 2 4 8        (BC      )$       reduce by A -> BC
0 2 3          (A       )$       shift 7
0 2 3 7        (A)      $        reduce by S -> (A)
0 1            S        $        accept
```

# 4 Ítem 3: definición dirigida por la sintaxis

## 4.1 Gramática

La SDD calcula el valor de un número escrito en una base arbitraria. La cadena $b{:}d_1 \cdots d_k$ significa "los dígitos $d_1 \cdots d_k$ leídos en base $b$", donde la base $b$ se escribe en decimal: `8:17` vale $1 \cdot 8 + 7 = 15$ y `12:11` vale 13.

```
4
S -> N:N
N -> DR
R -> DR e
D -> 0 1 2 3 4 5 6 7 8 9
```

Cumple G1 (convenciones del §2.2; es LL(1) y SLR(1): FIRST($DR$) son los dígitos y FOLLOW($R$) = $\{:, \$\}$), G2 (4 no terminales, 14 producciones, $R$ es recursivo) y G3: no es ninguna de las SDD del libro ni del enunciado. A diferencia de las SDD de la Fig. 5.4 (expresiones) o de los números binarios, aquí el valor de un subárbol (la base, $N_1.val$) se convierte en atributo heredado del subárbol hermano ($N_2.base$), y el valor se acumula de izquierda a derecha con la regla de Horner mediante un heredado ($R.acc$).

### 4.2 Tabla de atributos

| Símbolo | Atributo | Clase | Tipo | Significado |
|---|---|---|---|---|
| $S$ | *val* | sintetizado | entero / error | valor del número (atributo principal); error si algún dígito no es menor que su base |
| $S$ | *ok* | sintetizado | booleano | los dos números están bien formados |
| $N$ | *base* | heredado | entero | base en la que se leen los dígitos de $N$ |
| $N$ | *val* | sintetizado | entero | valor del número |
| $N$ | *ok* | sintetizado | booleano | todos los dígitos de $N$ son menores que la base |
| $R$ | *base* | heredado | entero | la misma base, pasada a lo largo de la lista |
| $R$ | *acc* | heredado | entero | valor de los dígitos a la izquierda de $R$ (Horner) |
| $R$ | *val* | sintetizado | entero | valor de toda la lista de dígitos |
| $R$ | *ok* | sintetizado | booleano | todos los dígitos de la lista son menores que la base |
| $D$ | $v$ | sintetizado | entero | valor numérico del dígito |
| dígitos y `:` | *lexval* | sintetizado | carácter | el propio carácter (lo da el "escáner") |

### 4.3 Reglas semánticas

En el formato de la Fig. 5.1 (los símbolos repetidos llevan subíndice):

| Producción | Reglas semánticas |
|---|---|
| $S \to N_1 : N_2$ | $N_1.base = 10$<br>$N_2.base = N_1.val$<br>$S.val = N_2.val$ si $N_1.ok \wedge N_2.ok$, si no *error*<br>$S.ok = N_1.ok \wedge N_2.ok$ |
| $N \to D\,R$ | $R.base = N.base$<br>$R.acc = D.v$<br>$N.val = R.val$<br>$N.ok = (D.v < N.base) \wedge R.ok$ |
| $R \to D\,R_1$ | $R_1.base = R.base$<br>$R_1.acc = R.acc \cdot R.base + D.v$<br>$R.val = R_1.val$<br>$R.ok = (D.v < R.base) \wedge R_1.ok$ |
| $R \to \varepsilon$ | $R.val = R.acc$<br>$R.ok = true$ |
| $D \to d \quad (d \in \{0,\ldots,9\})$ | $D.v = \mathrm{int}(d.lexval)$ |

Comprobación de §5.1.1: en cada producción, cada sintetizado de la cabeza y cada heredado de un no terminal del cuerpo se define exactamente una vez (por ejemplo, en $R \to DR_1$ se definen $R.val$, $R.ok$, $R_1.base$ y $R_1.acc$); ninguna regla usa atributos de símbolos fuera de su producción; $S$ no tiene heredados. El validador estático del programa verifica lo mismo.

### 4.4 Ejemplo resuelto: la cadena `2:101`

La cadena tiene 5 terminales y vale $1\cdot4+0\cdot2+1=5$. La Figura 2 es el árbol anotado; los números $\#k$ son los identificadores de los nodos en preorden, que usamos para nombrar los vértices del grafo.

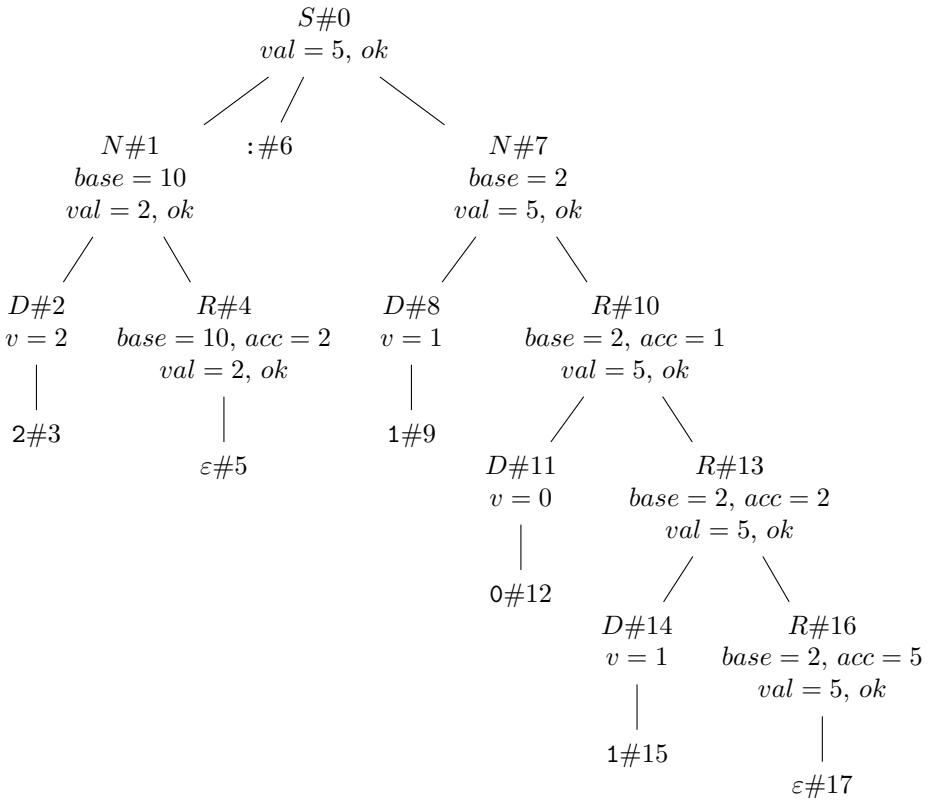



Figura 2: Árbol de análisis anotado de `2:101` ($ok = true$ en todos los nodos).

La Figura 3 es el grafo de dependencias (§5.2.1): un vértice por cada atributo de cada nodo (los heredados a la izquierda del nodo y los sintetizados a la derecha, como en la Fig. 5.7) y una arista desde cada atributo usado hacia el atributo definido. Lo más interesante es la arista $N\#1.val \to N\#7.base$, que lleva información de un subárbol a su hermano derecho, y la cadena $R\#10.acc \to R\#13.acc \to R\#16.acc \to R\#16.val \to R\#13.val \to R\#10.val \to N\#7.val \to S\#0.val$, que baja por la lista acumulando el valor y luego lo sube.

Un orden de evaluación válido es el orden topológico que produce el programa (`python3 main.py -sdd < examples/sdd_hand.txt`):

```
 1. N#1.base = 10       12. 1#9.lexval = '1'    23. R#16.base = 2
 2. 2#3.lexval = '2'    13. D#8.v = 1           24. R#16.ok = True
 3. D#2.v = 2           14. R#10.acc = 1        25. R#13.ok = True
 4. R#4.acc = 2         15. R#10.base = 2       26. R#10.ok = True
 5. R#4.base = 10       16. 0#12.lexval = '0'   27. N#7.ok = True
 6. R#4.ok = True       17. D#11.v = 0          28. S#0.ok = True
 7. N#1.ok = True       18. R#13.acc = 2        29. R#16.val = 5
 8. R#4.val = 2         19. R#13.base = 2       30. R#13.val = 5
 9. N#1.val = 2         20. 1#15.lexval = '1'   31. R#10.val = 5
10. :#6.lexval = ':'    21. D#14.v = 1          32. N#7.val = 5
11. N#7.base = 2        22. R#16.acc = 5        33. S#0.val = 5
```

Cada vértice aparece después de todos sus predecesores; por ejemplo $N\#7.base$ (paso 11) va después de $N\#1.val$ (paso 9), y $R\#16.acc = R\#13.acc \cdot R\#13.base + D\#14.v = 2 \cdot 2 + 1 = 5$ (paso 22) va después de los pasos 18, 19 y 21.

## 4.5 Clasificación

**No es S-atribuida**: según §5.2.3, una SDD es S-atribuida si todos sus atributos son sintetizados, y la nuestra tiene heredados ($N.base$, $R.base$, $R.acc$). De hecho, como el enunciado exige al menos

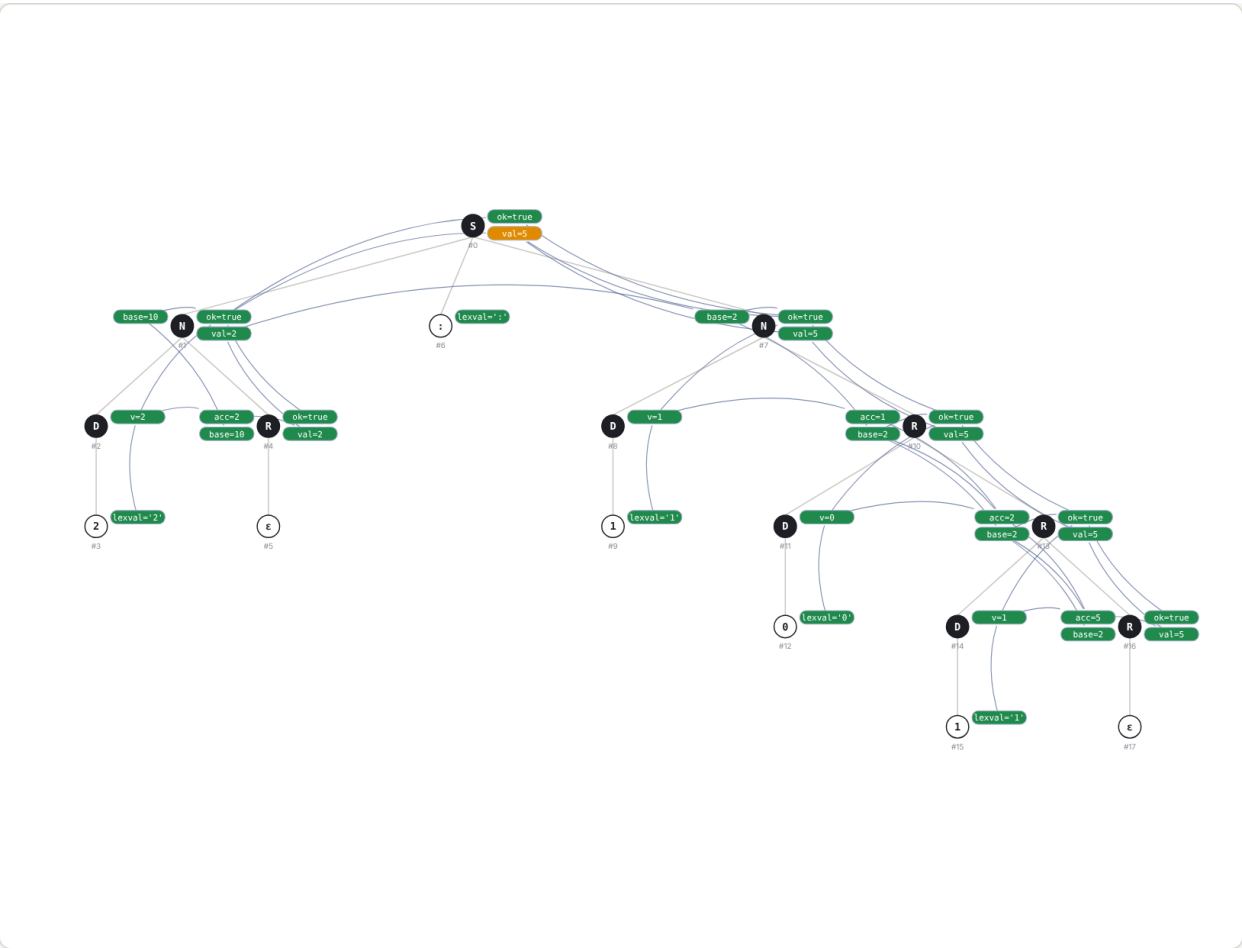



Figura 3: Grafo de dependencias de `2:101`, con todos los atributos evaluados (visualizador).

un heredado, ninguna SDD válida del proyecto puede ser S-atribuida.

**Sí es L-atribuida** (§5.2.4): todo atributo es sintetizado, o es heredado y su regla en $A \to X_1 \cdots X_n$ que define $X_i.a$ usa solo (a) heredados de la cabeza $A$, (b) atributos de $X_1, \ldots, X_{i-1}$ o (c) atributos de $X_i$ sin formar ciclos. Revisamos cada heredado:

- $S \to N_1 : N_2$: $N_1.base = 10$ no usa nada; $N_2.base = N_1.val$ usa un atributo de $N_1$, que está a la izquierda de $N_2$ (b).
- $N \to DR$: $R.base = N.base$ usa un heredado de la cabeza (a); $R.acc = D.v$ usa $D$, a la izquierda de $R$ (b).
- $R \to DR_1$: $R_1.base = R.base$ (a); $R_1.acc = R.acc \cdot R.base + D.v$ usa heredados de la cabeza (a) y $D$, a la izquierda de $R_1$ (b).

Las demás reglas definen atributos sintetizados. La función `classify` del programa da el mismo resultado (`S-attributed: no, L-attributed: yes`). Aunque sea L-atribuida, el evaluador usa el método general del grafo de dependencias (Step 5).

## 4.6 Pruebas

Salida de `python3 main.py -sdd < examples/sdd_tests.txt` (se muestra solo el resultado; el programa también imprime el árbol y el orden para cada cadena):

| Cadena | Por qué | Resultado |
|---|---|---|
| `2:0` | la más corta del lenguaje (3 terminales) | $S.val = 0$ |
| `8:17` | caso típico | $S.val = 15$ |
| `12:11` | base de dos dígitos (usa $N_2.base = N_1.val$) | $S.val = 13$ |
| `9:9` | dígito no menor que la base | $S.val = error$, $S.ok = false$ |
| `8:1x` | no está en el lenguaje | `no` |
| `2:1`$^{40}$ | recursión profunda (40 nodos $R$ anidados) | $S.val = 2^{40} - 1 = 1099511627775$ |

Las pruebas automáticas (`tests/test_sdd.py`) verifican además que las cadenas den el mismo árbol y el mismo valor con los dos analizadores, que el orden producido es realmente topológico (cada arista va de un vértice anterior a uno posterior) y una cadena con 60 dígitos.

### 4.7 Experimento de circularidad

En `sdd/my_sdd_circular.py` cambiamos una sola regla:

$$N \to DR: \qquad R.acc = D.v \quad \rightsquigarrow \quad R.acc = R.val + D.v.$$

La SDD sigue siendo sintácticamente válida (el validador la acepta: $R.acc$ sigue siendo un heredado de un símbolo del cuerpo, definido una vez), pero ahora $R.acc$ depende de $R.val$, y $R.val$ depende —bajando por la lista hasta $R \to \varepsilon$, donde $R.val = R.acc$— de $R.acc$. La clasificación automática ya la marca como no L-atribuida, y el evaluador detecta el ciclo en el grafo de cada árbol:

```
$ python3 main.py --sdd --circular < examples/sdd_hand.txt
SDD: number in an arbitrary base (circular)
Parser: LL(1)
S-attributed: no   L-attributed: no
  (not S-attributed: there are inherited attributes)
  (not L-attributed: in N -> DR, R.acc uses R.val, a synthesized attribute of the same
   symbol (may be circular))

String: 2:101
SDD is circular: R#16.acc -> R#16.val -> R#13.val -> R#10.val -> R#10.acc -> R#13.acc
                 -> R#16.acc
```

## 5 Pruebas de los Ítems 1 y 2

`tests/test_parsers.py` (ejecutar con `python3 -m unittest discover -s tests`) compara con resultados del libro y del enunciado:

- FIRST y FOLLOW de la gramática (4.28) (Ejemplo 4.30), la tabla de la Fig. 4.17 y los movimientos de la Fig. 4.21.
- El autómata LR(0) de la Fig. 4.31 (12 estados con la misma numeración), la tabla SLR de la Fig. 4.37 (36 entradas ACTION y la tabla GOTO completa) y los 14 movimientos de la Fig. 4.38.
- La salida exacta de los Ejemplos 1, 2 y 3 del enunciado y los cuatro casos de clasificación; para "LL(1) pero no SLR(1)" usamos $S \to AaAb \mid BbBa$, $A \to \varepsilon$, $B \to \varepsilon$, que tiene conflictos reducción/reducción en el estado 0.
- Casos borde: $S \to \varepsilon$, la cadena `e`, símbolos que no están en la gramática (incluido `$`), alternativas duplicadas, la primera línea no es la de $S$, finales de línea de Windows, fin de archivo sin línea vacía y un FOLLOW que necesita varias pasadas.

Las 36 pruebas pasan con Python 3.11 y 3.13.

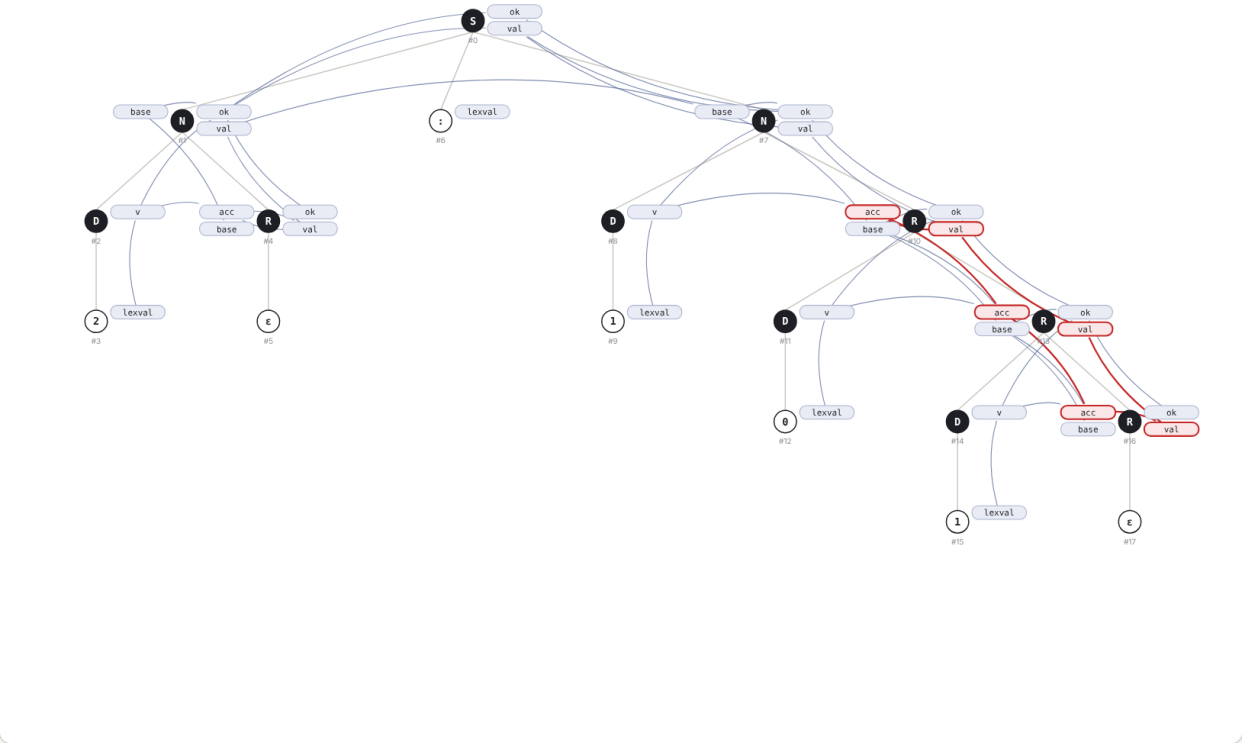



Figura 4: Grafo de dependencias de `2:101` con la SDD mutada; el ciclo está en rojo.

# 6 Problemas encontrados y cómo los resolvimos

1. **Orden de impresión de los conjuntos.** El enunciado pide los terminales en orden ASCII, luego `e` y luego `$`. Como `$` (ASCII 36) es menor que las letras y los dígitos, un `sorted` directo lo ponía primero. Se ordenan solo los terminales y se agregan `e` y `$` al final. Queda una ambigüedad: si se imprimen todas las líneas `First` y luego todas las `Follow`, o intercaladas por no terminal; lo dejamos en una constante (`SETS_GROUPED`) para cambiarlo según lo que diga el profesor.

2. **Condición LL(1) frente a conflictos en la tabla.** Implementamos la definición de §4.4.3 y además verificamos que la tabla no tenga entradas múltiples. Pensábamos que las dos pruebas siempre coinciden, pero no es así cuando un no terminal es inalcanzable desde $S$: si tiene dos alternativas que derivan $\varepsilon$, FOLLOW es vacío, la condición 2 falla y la tabla no tiene conflicto. Exigimos las dos.

3. **El símbolo $S'$.** Como los símbolos de la entrada son de un carácter, cualquier letra que usáramos para el nuevo símbolo inicial podía chocar con uno real. Usamos la cadena `"S'"`, que no puede aparecer en la entrada.

4. **Numeración de estados.** Para comparar con la Fig. 4.31 y con nuestros cálculos a mano, el orden de los estados tenía que ser predecible. Recorremos los estados en BFS y, dentro de cada uno, los símbolos en el orden en que aparecen después del punto.

5. **Árbol desde el analizador SLR(1).** El algoritmo 4.44 no construye el árbol. Primero pensamos usar una pila de nodos durante el análisis, pero preferimos que ambos analizadores

devuelvan su derivación: invertir las reducciones da la derivación por la derecha, y basta expandir los hijos de derecha a izquierda (consumiendo la entrada desde el final). Una prueba verifica que los dos analizadores producen el mismo árbol.

6. **Símbolos repetidos en una producción.** En $S \to N : N$ hay que distinguir $N_1$ de $N_2$. Las reglas se escriben con los subíndices del libro y una función los traduce a posiciones del cuerpo.

7. **Hojas $\varepsilon$.** El nodo $\varepsilon$ es un hijo en el árbol pero no un símbolo del cuerpo de la producción; al instanciar las reglas se omite para que las posiciones coincidan.

8. **Clasificación de la SDD mutada.** La primera versión de `classify` aceptaba que un heredado de $X_i$ usara cualquier atributo de $X_i$ (condición (c) de §5.2.4) y por eso declaraba L-atribuida la SDD circular. La condición (c) exige que no haya ciclos; como un sintetizado de $X_i$ puede depender de sus heredados, ahora solo aceptamos otros heredados de $X_i$.

9. **Valor de una cadena con dígitos inválidos.** Con `9:9` el valor principal salía 9 aunque el número estuviera mal formado. Hicimos que $S.val$ dependa también de $N_1.ok$ y $N_2.ok$.

10. **Un valor esperado mal contado en las pruebas.** Habíamos escrito que la Fig. 4.37 tiene 41 entradas ACTION; al contar de nuevo fila por fila son 36, que es lo que da el programa.

## Referencias


[1] A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman. *Compilers: Principles, Techniques, and Tools*, 2.ª ed. Addison-Wesley, 2006.